Linux Shell Scripting Lab Instructions

- All scripts must start with #!/bin/bash
- Use comments (#) to explain your logic.
- Test your scripts on a Linux terminal before submitting.

Q1

Scenario: A new engineer joins your team. Write a script that asks for their name, department, and years of experience, then prints a formatted welcome message.

Your Tasks:

1. Prompt the user to enter: name, department, and years of experience
2. Store each value in a properly named variable
3. Print: 'Welcome [name]! You joined the [dept] team with [X] years of experience.'
4. If years of experience equals 0, print: 'Welcome aboard as a fresher!' instead

```
[root@localhost ~]# su - mina
Last login: Tue Mar 17 22:20:52 EET 2026 on pts/0
[mina@localhost ~]$
[mina@localhost ~]$ touch welcome.sh
[mina@localhost ~]$ ls
passwd_copy  welcome.sh
[mina@localhost ~]$ sudo chmod +x welcome.sh
[sudo] password for mina:
[mina@localhost ~]$ ls
passwd_copy  welcome.sh
[mina@localhost ~]$ ls -l welcome.sh
-rwxr-xr-x. 1 mina mina 0 Mar 17 22:51 welcome.s
h
[mina@localhost ~]$ ./welcome.sh
[mina@localhost ~]$ vi welcome.sh
[mina@localhost ~]$ vi welcome.sh
[mina@localhost ~]$ cat welcome.sh
#!/bin/bash
# Q1: Welcome new engineer

# Prompt for name
read -p "Enter your name: " NAME
# Prompt for department
read -p "Enter your department: " DEPT
# Prompt for years of experience
read -p "Enter years of experience: " YEARS

# Check if user is a fresher
if [ "$YEARS" -eq 0 ]; then
    echo "Welcome aboard as a fresher!"
else
    echo "Welcome $NAME! You joined the $DEPT team with $YEARS years of experience."
fi

[mina@localhost ~]$ bash welcome.sh
Enter your name: mina
Enter your department: is
Enter years of experience: 0
Welcome aboard as a fresher!
[mina@localhost ~]$
```

Q2

Scenario: Before running a backup job, the script must verify that the source folder and a config file both exist on the system.

Your Tasks:

5. Accept two arguments: $1 = folder path, $2 = config file path
6. Check if the folder exists (-d); print 'OK: folder found' or 'ERROR: folder missing'
7. Check if the config file exists (-f); print OK or ERROR accordingly
8. If either is missing, exit the script with exit 1
9. If both exist, print: 'All checks passed. Ready to run.'

Starter Code:

```
#!/bin/bash# Usage:
./q2.sh /path/to/folder
/path/to/config.cfg
FOLDER=$1CONFIG=$2
```

```
[mina@localhost ~]$ vi welcome.sh
[mina@localhost ~]$ cat welcome.sh
#!/bin/bash
# Q2: Check folder and config file before backup

FOLDER=$1
CONFIG=$2

# Check folder existence
if [ -d "$FOLDER" ]; then
    echo "OK: folder found"
else
    echo "ERROR: folder missing"
    exit 1
fi

# Check config file existence
if [ -f "$CONFIG" ]; then
    echo "OK: config file found"
else
    echo "ERROR: config file missing"
    exit 1
fi

echo "All checks passed. Ready to run."
[mina@localhost ~]$ bash welcome.sh
ERROR: folder missing
[mina@localhost ~]$
```

Q3

Scenario: Your server produces a log file called server.log. Write a script to extract useful information from it quickly.

Your Tasks:

10. Count how many lines contain the word ERROR (use grep -c)
11. Save all ERROR lines to a file called errors_today.txt
12. Count unique IP addresses — assume each line starts with an IP address (hint: awk '{print $1}')
13. Print a summary: 'Total errors: X | Unique IPs: Y'

Starter Code:

```
#!/bin/bashLOG='server.log'# Hint: sort | uniq | wc -l counts unique values
```

```
[mina@localhost ~]$ cat welcome.sh
#!/bin/bash

# Define log file
LOG='server.log'

# 10. Count lines containing the word ERROR
ERROR_COUNT=$(grep -c "ERROR" "$LOG")

# 11. Save all ERROR lines to errors_today.txt
grep "ERROR" "$LOG" > errors_today.txt

# 12. Count unique IP addresses (assume IP is the first field)
UNIQUE_IPS=$(awk '{print $1}' "$LOG" | sort | uniq | wc -l)

# 13. Print summary
echo "Total errors: $ERROR_COUNT | Unique IPs: $UNIQUE_IPS"
[mina@localhost ~]$ touch server.log
[mina@localhost ~]$ echo "192.168.1.4 INFO User connected" >> serv
er.log
[mina@localhost ~]$ echo "192.168.1.5 ERROR Timeout occurred" >> s
erver.log
[mina@localhost ~]$ echo "192.168.1.6 INFO Backup completed" >> se
rver.log
[mina@localhost ~]$ echo "192.168.1.3 ERROR Disk almost full" >> s
erver.log
[mina@localhost ~]$ echo "192.168.1.7 INFO Server rebooted" >> ser
ver.log
[mina@localhost ~]$ echo "192.168.1.5 ERROR Failed authentication"
 >> server.log
[mina@localhost ~]$ echo "192.168.1.8 INFO Configuration updated"
>> server.log
[mina@localhost ~]$ echo "192.168.1.2 ERROR Network unreachable" >
> server.log
[mina@localhost ~]$ bash welcome.sh
Total errors: 4 | Unique IPs: 8
[mina@localhost ~]$
```

Q4

Scenario: A data pipeline needs 7 daily report folders (day1 through day7), each containing an empty report file.

Your Tasks:

14. Use a for loop from 1 to 7
15. Inside each iteration: create a folder called dayN/ and an empty file report_dayN.txt inside it
16. Print: 'Created: day1/report_day1.txt' for each iteration
17. After the loop, use find to count and print the total number of files created

```
[mina@localhost ~]$ echo >  welcome.sh
[mina@localhost ~]$ vi welcome.sh
[mina@localhost ~]$ cat welcome.sh
#!/bin/bash

# Loop from 1 to 7
for i in {1..7}
do
    # Create folder dayN
    mkdir -p "day$i"

    # Create empty file inside it
    touch "day$i/report_day$i.txt"

    # Print confirmation
    echo "Created: day$i/report_day$i.txt"
done

# Count total files created
total=$(find . -type f -name "report_day*.txt" | wc -l)

echo "Total files created: $total"
[mina@localhost ~]$ bash welcome.sh
Created: day1/report_day1.txt
Created: day2/report_day2.txt
Created: day3/report_day3.txt
Created: day4/report_day4.txt
Created: day5/report_day5.txt
Created: day6/report_day6.txt
Created: day7/report_day7.txt
Total files created: 7
[mina@localhost ~]$
```

Q5

Scenario: Write a monitoring script that checks disk usage every 5 seconds and stops when usage is below a safe threshold.

Your Tasks:

18. Use a while loop that runs as long as disk usage of / is above 20%
19. Inside the loop: print current usage % and the timestamp using date
20. Use sleep 5 between checks
21. When usage drops below 20%, print: 'Disk usage is normal. Exiting.'
22. Add a safety counter — if the loop runs more than 10 times, break and print a warning

Starter Code:

```
#!/bin/bashCOUNT=0#
Hint: df -h / | awk
'NR==2 {print $5}' | tr
-d '%'
```

```
[mina@localhost ~]$ vi welcome.sh
[mina@localhost ~]$ bash welcome.sh
Tue Mar 17 11:38:53 PM EET 2026: Current disk usage is 7%
Disk usage is normal. Exiting.
[mina@localhost ~]$
```

Q6

Scenario: Write a script with two functions that process employee data. Functions must use local variables so they do not overwrite each other.

Your Tasks:

23. Function get_grade(salary): returns 'Senior' if salary >= 90000, 'Mid' if >= 70000, else
'Junior'
24. Function format_employee(name, dept, salary): prints a formatted line and calls get_grade()
25. Call format_employee for at least 3 employees with different salaries
26. Demonstrate that local variables work: define RESULT inside each function with local
and prove the global RESULT is unchanged after both calls

```
[mina@localhost ~]$ vi welcome.sh
[mina@localhost ~]$ bash welcome.sh
Name: Mina | Dept: HR | Salary: 95000 | Grade: Senior
Name: Shafik | Dept: IT | Salary: 75000 | Grade: Mid
Name: hazem | Dept: Sales | Salary: 50000 | Grade: Junior
Global RESULT after function calls: GLOBAL_VALUE
[mina@localhost ~]$
```

Q7

Scenario: You have a file sales.csv with columns: region,product,quantity,price. Write a script using awk to analyze it.

Your Tasks:

27. Print all rows where quantity > 100
28. Calculate total revenue per region (revenue = quantity * price) and print a summary
29. Find the product with the highest price
30. Use BEGIN to print a report header and END to print total revenue across all rows

Starter Code:

```
#!/bin/bashFILE='sales.
csv'# Create sample
data first:# printf
'region,product,quantit
y,price\nNorth,Laptop,5
0,1200\nSouth,Phone,200
,800
```

```
[mina@localhost ~]$ echo >welcome.sh
[mina@localhost ~]$ vi welcome.sh
[mina@localhost ~]$ bash welcome.sh
===== Sales Report =====
High Quantity Row: South,Phone,200,800
High Quantity Row: East,Tablet,150,600
High Quantity Row: North,Phone,120,700

--- Revenue per Region ---
South: 160000
East: 90000
North: 144000
West: 24000

Highest Priced Product: Laptop at 1200

Total Revenue: 418000
[mina@localhost ~]$ ls
day1  day3  day5  day7                passwd_copy  server.log
day2  day4  day6  errors_today.txt    sales.csv    welcome.sh
[mina@localhost ~]$
```

Q8

Scenario: You received a messy config file from a client. It has wrong values, comment lines, and blank lines. Clean it with sed.

Your Tasks:

31. Remove all lines starting with # (comment lines)
32. Remove all blank/empty lines
33. Replace all occurrences of 'localhost' with '192.168.1.10'
34. Replace 'port=8080' with 'port=443'
35. Save the cleaned output to config_clean.conf
36. Print: 'Done. Lines before: X Lines after: Y' (use wc -l)

Starter Code:

```
#!/bin/bashINPUT='confi
g.conf'OUTPUT='config_c
lean.conf'# Hint: chain
multiple sed commands
using -e# Hint: wc -l <
filename counts lines
in a file
```

```
[mina@localhost ~]$ echo >welcome.sh
[mina@localhost ~]$ vi welcome.sh
[mina@localhost ~]$ bash welcome.sh
Done. Lines before: 7 Lines after: 3
[mina@localhost ~]$ cat config_clean.conf
host=192.168.1.10
port=443
mode=dev
[mina@localhost ~]$
```

Q9

Scenario: You have employees.csv with columns: id,name,department,salary,status. Use cut and pipes to extract data.

Your Tasks:

37. Print only the name and department columns (skip the header with tail -n +2)
38. List only active employees (status = active) — combine grep and cut
39. Find the highest salary using cut and sort -n
40. Count employees per department using cut, sort, and uniq -c
41. Save a report of name + salary sorted by salary descending to report.txt

Starter Code:

```
#!/bin/bashFILE='employ
ees.csv'# Hint: tail -n
+2 skips the header
line
```

```
[mina@localhost ~]$ echo >welcome.sh
[mina@localhost ~]$ vi welcome.sh
[mina@localhost ~]$ bash welcome.sh
---- Task 37: Name & Department ----
Alice,HR
Bob,IT
Charlie,Sales
David,IT
Eve,HR

---- Task 38: Active Employees ----
Alice,HR,active
Charlie,Sales,active
David,IT,active
Eve,HR,active

---- Task 39: Highest Salary ----
120000

---- Task 40: Employees per Department ----
      2 HR
      2 IT
      1 Sales

---- Task 41: Report (Name + Salary Desc) ----
Report saved to report.txt
[mina@localhost ~]$ ls
config_clean.conf  day1  day3  day5  day7          errors_today.txt  report.txt  server.log
config.conf        day2  day4  day6  employees.csv  passwd_copy       sales.csv   welcome.sh
[mina@localhost ~]$
```

Q10

Scenario: Write a complete system health check script combining everything learned. The ops team will run this every morning.

Your Tasks:

42. Function check_disk(): alert if / usage > 80% (use df -h)
43. Function check_memory(): alert if free RAM < 500mb (use free -m)
44. Function check_process(name): check if a process is running using pgrep — print RUNNING or STOPPED
45. Call all functions from main; pass 'sshd' and 'cron' to check_process
46. Append all ALERT messages to health.log with a timestamp
47. Print a final summary line: 'X checks passed Y alerts raised'

```bash
#!/bin/bash

LOGFILE="health.log"
ALERTS=0
PASSES=0

# Function: Check Disk Usage
check_disk() {
    usage=$(df -h / | awk 'NR==2 {print $5}' | tr -d '%')

    if [ "$usage" -gt 80 ]; then
        echo "$(date): ALERT - Disk usage is ${usage}%" >> "$LOGFILE"
        ((ALERTS++))
    else
        ((PASSES++))
    fi
}

# Function: Check Memory
check_memory() {
    free_mem=$(free -m | awk 'NR==2 {print $7}')  # available memory

    if [ "$free_mem" -lt 500 ]; then
        echo "$(date): ALERT - Low memory: ${free_mem}MB available" >> "$LOGFILE"
        ((ALERTS++))
    else
        ((PASSES++))
    fi
}

# Function: Check Process
check_process() {
    local pname=$1

    if pgrep "$pname" > /dev/null; then
        echo "$pname is RUNNING"
        ((PASSES++))
    else
        echo "$pname is STOPPED"
        echo "$(date): ALERT - Process $pname is not running" >> "$LOGFILE"
        ((ALERTS++))
    fi
}

# Main execution
main() {
    check_disk
```

```
[mina@localhost ~]$ bash welcome.sh
sshd is RUNNING
cron is RUNNING
Summary: 4 checks passed, 0 alerts raised
[mina@localhost ~]$ vi welcome.sh
```